

Introduction to .NET Core & ASP.NET Core

Module 1: Introduction to .NET Core

What is .NET Core?

.NET Core (now evolved into modern .NET) is a **cross-platform, open-source framework** developed by **Microsoft** for building modern applications.

Supports:

- Windows
 - Linux
 - macOS
-

Benefits of .NET Core

- Cross-platform development
 - High performance
 - Open-source and community-driven
 - Scalable for cloud-based applications
 - Lightweight and modular
-

What's New in .NET Core?

- Improved performance over .NET Framework
 - Built-in Dependency Injection
 - Unified platform (now .NET 5+)
 - Better support for microservices and containers
 - CLI-based development
-

.NET Core vs .NET Framework

Feature	.NET Core	.NET Framework
Platform	Cross-platform	Windows only
Performance	High	Moderate
Open Source	Yes	Limited
Deployment	Flexible	System-dependent
Use Case	Modern apps, cloud	Legacy applications

First .NET Core Application

Step 1: Install SDK

Download from official website

Step 2: Create Project

```
dotnet new console -n FirstApp
```

```
cd FirstApp
```

Step 3: Run Application

```
dotnet run
```

Building .NET Core Applications

- Use CLI or Visual Studio
- Project structure includes:
 - Program.cs
 - .csproj file

Build command:

dotnet build

Module 2: Middleware and Static Files

Understanding Middleware

Middleware are components in the **request pipeline** that handle HTTP requests and responses.

Examples:

- Authentication
 - Logging
 - Routing
-

Configuring Middleware

```
app.UseRouting();
```

```
app.UseAuthorization();
```

```
app.MapGet("/", () => "Hello World");
```

Serving Static Files

Enable static file support:

```
app.UseStaticFiles();
```

Files are served from:

- wwwroot folder

Examples include:

- HTML
- CSS
- JavaScript

Security Considerations with Static Files

- Restrict access to sensitive files
 - Use authentication where required
 - Avoid exposing server directories
-

Module 3: Introduction to Razor Pages

Overview of Razor Pages

Razor Pages provide a **page-based programming model** for building web applications in ASP.NET Core.

Advantages over Traditional MVC

- Simpler structure
 - Reduced code complexity
 - Suitable for small to medium applications
 - Faster development cycle
-

Creating Razor Pages Project

```
dotnet new webapp -n RazorApp
```

```
cd RazorApp
```

```
dotnet run
```

Folder Structure and Naming

```
/Pages
```

```
Index.cshtml
```

```
Index.cshtml.cs
```

```
/wwwroot
```

Naming conventions:

- `.cshtml` → UI
- `.cshtml.cs` → backend logic

Module 4: Razor Syntax and PageModel

Razor Syntax Basics

Use `@` to switch from HTML to C#:

```
<p>@DateTime.Now</p>
```

Mixing HTML and Server-side Code

```
@{  
    var name = "John";  
}  
  
<h1>Hello @name</h1>
```

Understanding the PageModel Class

The PageModel handles backend logic:

```
public class IndexModel : PageModel  
{  
    public void OnGet()  
    {  
    }  
}
```

```
// Logic here  
  
}  
  
}
```

Property Binding

```
[BindProperty]  
  
public string Name { get; set; }
```

Handling Requests

Method	Purpose
OnGet()	Handles GET request
OnPost()	Handles POST request

Summary

- .NET Core is a cross-platform development framework
 - Middleware handles request processing
 - Razor Pages simplify web development
 - PageModel manages backend logic
-

Next Steps for Learners

- Build a simple Razor Pages application
- Implement form handling using POST
- Serve static files
- Explore middleware customization

